

Smart contract audit

rain.lib.memkv

Content

Project description	3
Executive summary	3
Reviews	3
Security findings	6
**** Critical	6
*** High	6
H01 - Data Corruption with Memory > 64KB	6
** Medium	7
* Low	7
L01 - Item Count Overflow (Silent Wrap-Around)	7
Informational	8
Approach and methodology	9



Project description

This repository contains a Solidity library that implements an optimized in-memory key/value store using Yul for gas efficiency. It utilizes a hash map structure backed by 15 internal linked lists to provide roughly $O(1)$ time complexity for get and set operations.

The library is designed to handle key/value pairs (both bytes32) within the scope of a single transaction. It features a custom MemoryKV pointer type that tracks the head of the lists and the total word count, allowing for efficient $O(1)$ memory allocation when exporting the store. A specialized toBytes32Array function allows the entire data structure to be snapshotted into a flat uint256[] array using a gas-optimized bisection algorithm. This library is intended for complex transaction logic requiring temporary, high-performance structured data storage that does not require persistent contract storage.

Executive summary

Type	Library
Languages	Solidity
Methods	Architecture Review, Manual Review, Unit Testing, Functional Testing, Automated Review
Documentation	README.md
Repositories	https://github.com/rainlanguage/rain.lib.memkv

Reviews

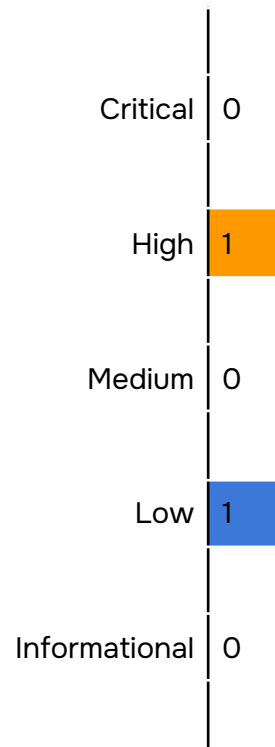
Date	Repository	Commit
10/02/26	rain.lib.memkv	7dc72b7f0a3061d9a5ad8e1950ebad69ee96c2e2
13/02/26	rain.lib.memkv	83e607990be8b3e06549338043c6b18f430f6bd2
22/09/26	rain.lib.memkv	05dd78183e919872924ee061b855e29424bc43b4 tag sol-v0.2.0



Scope

Contracts
src/lib/LibMemoryKV.sol

Technical analysis and findings



Security findings

**** Critical

No critical severity issue found.

*** High

H01 - Data Corruption with Memory > 64KB

The LibMemoryKV library relies on a packed data structure where memory pointers are stored in 16-bit slots within a single uint256. This design assumes that the Solidity free memory pointer will never exceed 65535 (0xFFFF). This assumption is neither enforced in code nor documented anywhere in the library or its NatSpec.

However, if a transaction allocates more than 64KB of memory—a common occurrence in complex flows—the pointer requires more than 16 bits (e.g., 0x10000). In such cases, the library does not revert or signal an error and instead silently corrupts its internal state. This triggers two catastrophic failures:

1. Read Failure (Truncation): In `get()`, the library retrieves pointers using `and(..., 0xFFFF)`. A valid pointer at 0x10000 is read as 0x0000, returning incorrect data or failing to find the key.
2. Write Failure (Corruption): In `set()`, the library does not validate that the pointer fits in 16 bits. If the pointer is 0x10005 (17 bits), the top bit (1) bleeds into the adjacent bucket's slot. This corrupts neighboring linked lists or the length counter, rendering unrelated keys inaccessible.

Because no bounds check or revert is performed, this corruption is silent and persistent, making the failure extremely difficult to detect or debug.

Impact:

The library is fundamentally unsafe for any environment where memory usage is not strictly bounded below 64KB.

Path: `src/lib/LibMemoryKV.sol`

Recommendation: Refactor the MemoryKV structure to use 32-bit slots.

Current: 15 buckets (16-bit).

New: 7 buckets (32-bit).

This sacrifice in hash-map width is necessary to support standard Solidity memory addressing (up to 4GB). If the 16-bit constraint is non-negotiable, the library must revert in `set()` if the pointer exceeds 0xFFFF to prevent silent corruption.

Additionally, the documentation should be expanded to clearly describe all underlying



assumptions and limitations, including the requirement that total memory usage must remain below 64KB for correctness.

Found in: 7dc72b7f

Status: Fixed at 83e6079

**** Medium**

No medium severity issue found.

*** Low**

L01 - Item Count Overflow (Silent Wrap-Around)

The library tracks the total number of words stored in the structure using the top 16 bits of the MemoryKV pointer (bits 240–255). This counter is incremented by 2 for every new key inserted (1 word for key + 1 word for value).

Because the counter is limited to 16 bits, the maximum value it can hold is 65535. Since each item consumes 2 units of "length", the maximum number of items the store can track is 32767 (65535 / 2).

If a user inserts a 32768th item:

1. The length calculation `add(..., 2)` results in 65536 (0x10000).
2. When this new length is shifted back into position (`shl(0xf0, length)`), the 1 at the 17th bit is shifted out of the uint256 range ($240 + 16 = 256$), effectively resetting the length field to 0.

Impact:

The `toBytes32Array` function relies entirely on this length field to allocate the return array (`mload(0x40)`). If the counter has wrapped around, this function will return an empty or severely truncated array, resulting in data loss during export/snapshot operations, even though the data technically exists in memory.

In practice, this condition is likely to be reached only after the 16-bit pointer limitation has already been violated. However, if the pointer packing is refactored (e.g., to 32-bit slots), this overflow becomes reachable and results in silent truncation of exported data.

Path: `src/lib/LibMemoryKV.sol`



Recommendation: The library must either support a larger item count or strictly enforce the 16-bit limit (according provided documentation).

Found in: 7dc72b7f

Status: Fixed at 83e6079

Informational

No informational severity issue found.



Approach and methodology

To establish a uniform evaluation, we define the following terminology in accordance with the OWASP Risk Rating

Methodology:

	Likelihood Indicates the probability of a specific vulnerability being discovered and exploited in real-world scenarios
	Impact Measures the technical loss and business repercussions resulting from a successful attack
	Severity Reflects the comprehensive magnitude of the risk, combining both the probability of occurrence (likelihood) and the extent of potential consequences (impact)

Likelihood and impact are divided into three levels: High H, Medium M, and Low L. The severity of a risk is a blend of these two factors, leading to its classification into one of four tiers: Critical, High, Medium, or Low.

When we identify an issue, our approach may include deploying contracts on our private testnet for validation through testing. Where necessary, we might also create a Proof of Concept PoC to demonstrate potential exploitability. In particular, we perform the audit according to the following procedure:

	Advanced DeFi Scrutiny We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs
	Semantic Consistency Checks We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
	Security Analysis The process begins with a comprehensive examination of the system to gain a deep understanding of its internal mechanisms, identifying any irregularities and potential weak spots.